

分离式推理架构1，从DistServe谈起

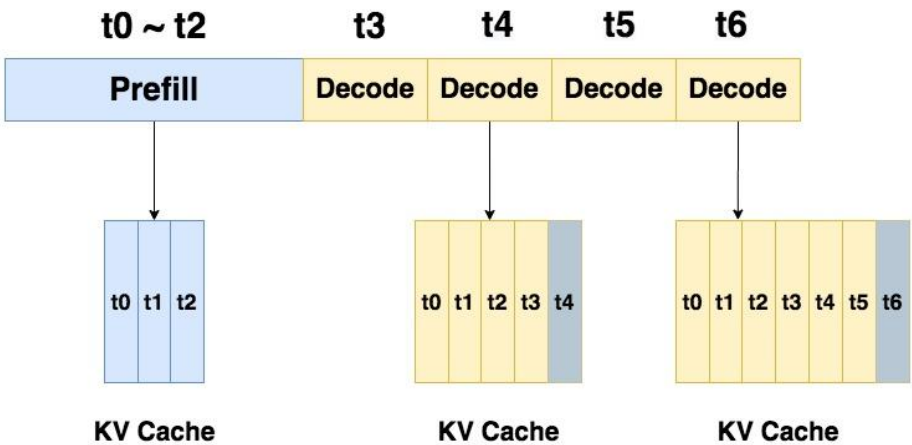
本篇作为该系列的第一篇，选择DistServe这个分离式架构入手，选择它的原因是：

- 这篇论文中通过各种实验和数学建模，很好论述了“为什么要用分离式架构”这一点。很适合初次了解这个架构的朋友阅读。这也是本文强调的重点
- 这篇论文代码是开源的，本文在写作时，也借鉴了开源代码的一些内容
- 调度策略比较简单（FCFS），也没有做抢占之类的操作（所以本文的重点也不会放在这里）

一、LLM推理的两阶段及评价指标

1.1 LLM推理的两阶段

LLM Inference



知乎 @猛猿

- **prefill阶段**：把整段prompt喂给模型做forward计算。prefill阶段结束后，模型产出第一个token（例如图中t3）
- **decode阶段**：一个token一个token地产出response。（例如图中逐一产出t4, t5, t6）

既然LLM的推理由这两个阶段组成，那么一个LLM系统的性能评估同样也和这两个阶段密切相关。解析来，我们来看两个分别用于评价这两阶段性能的指标。

1.2 prefill性能评估指标：TTFT

TTFT（Time To First Token），表示生成第1个token所用的时间，这是prefill阶段的重要评估指标之一。

在实践中，我们一般会人为设置一个TTFT SLO（Service Level Objective），这个指标是我们给系统

提的要求，只有系统达到了这个指标，我们才认为它的性能达标了。举例来说，假设我们定P90 TTFT SLO = 0.4s，那么这意味着我们对系统的要求是：“90%的request的TTFT值都必须<=0.4”

1.3 decode性能评估指标：TPOT

TPOT (Time Per Output Token) ，产出每一个response token所用的时间，这是decode阶段的重要评估指标之一。
同样，我们也有一个TPOT SLO指标，这是人为给系统定下的性能要求。

二、为什么需要做prefill和decode分离

在快速回顾LLM推理的两阶段及它们的评估指标后，现在我们来看一个重要的问题：为什么需要做PD分离？

2.1 分离架构速览

在一些推理框架中，prefill和decode是合并在一起的，你可以通俗理解成一块gpu上我既做prefill又做decode。我们以vllm框架为例：在一张卡上它的prefill和decode是交替进行的，即vllm的调度器会根据存储使用情况、请求池中的请求状态等要素，决定在模型推理的每1个阶段是做prefill还是做decode。

那分离式的框架长什么样呢？我们直接来看DistServe提供的一张架构简图：

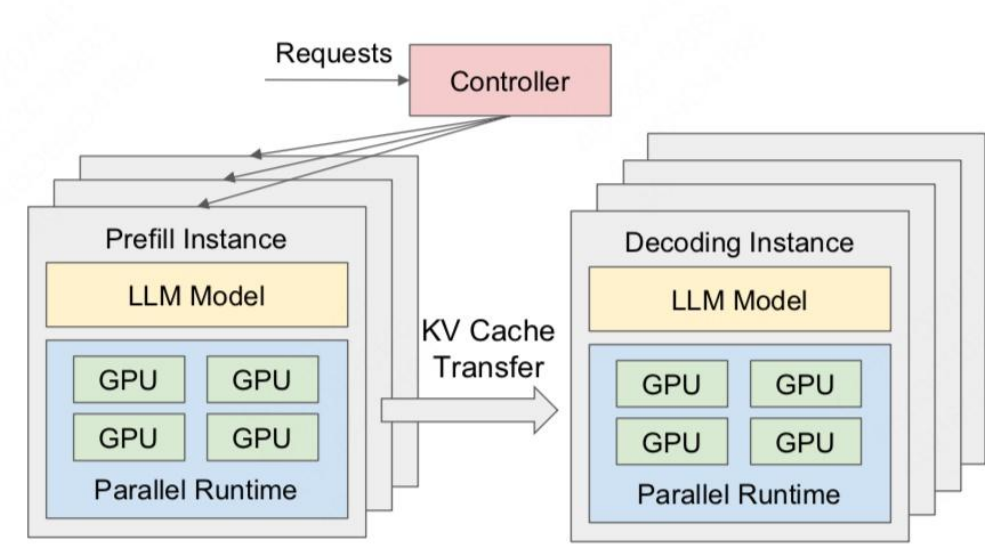


Figure 6: DistServe Runtime System Architecture 知乎 @猛猿

从这张架构简图中，我们不难发现，在分离式框架中prefill和decode不再共享一块/一群gpu，而是你做你的prefill，我做我的decode。prefill实例计算完毕后，把产出的KV Cache传送到decode实例，然后decode实例继续做推理。这里我们再明确一些术语：

- **Prefill instance: prefill实例。** 一个prefill实例中包含一个完整的模型，它可能占据1或多块gpu。
- **Decode instance: decode实例。** 定义同上，只是它与prefill实例不再共享gpu。

介绍到这里，你肯定开始疑惑了：**从直觉上看，分离式架构相比于合并式架构，多加载了模型副本（耗显存），同时还涉及到gpu间的KV Cache传输（耗时间），怎么看都好像比不上合并式架构，那么我们做分离的原因何在呢？**

接下来，我们就来回答这个问题。

2.2 TTFT和TPOT间的balance

假设你现在有一个requests pool，里面装了状态不同的requests，有正在等待做prefill的，也有刚做完prefill等待做decode的，而你面前只有1张GPU。由于prefill和decode在做fwd阶段的差异性，如果不考虑特殊的优化方法，在模型每个推理阶段，你要么全做prefill，要么全做decode。**现在你需要做一个决策：在下1个推理时刻，我该做prefill，还是做decode？**

你观察了一下requests pool，发现里面塞满了等待做prefill的请求，你心想“要不还是赶紧多做些prefill，好歹给用户返回第一个token，防止他们生气了直接不用我的产品了”。所以你在接下来的时刻中一直在做prefill，**由于gpu资源是有限的，你做了prefill就无法同时做decode了，你其实是在牺牲TPOT保全TTFT，反之亦然。**

所以这时，你产生了一个疑问：**当prefill和decode合并在一起的时候，似乎无论调度策略怎么调整，TTFT和TPOT这两个指标都存在强耦合关系。因此如果我把这两个阶段分开，让TTFT和TPOT各自独立发展优化，会不会得到一个更好的结果？**

DistServe做了一个实验来验证你的猜想，接下来我们就来看这个实验结果。

2.3 分离VS合并：一个实验结果

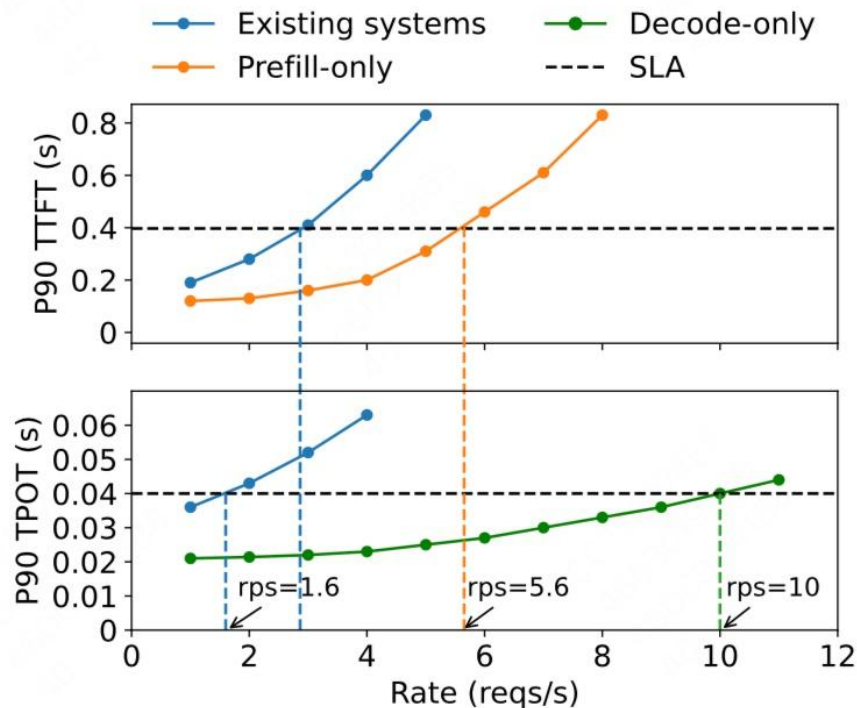


Figure 1: Performance when serving an LLM with 13B parameters under a synthetic workload with input length = 512 and output length = 64 on one NVIDIA 80GB A100. *Upper:* The P90 time-to-first-token (TTFT) latency comparing existing systems vs. a system serving only the prefill phase. *Down:* The P90 time-per-output-token (TPOT) latency comparing existing systems vs. a system serving only the decoding phase.

实验配置：

- 1张A100 80G的gpu
- 13B LLM, input_length = 512, output_length = 64

SLO (人为定义的系统性能达标要求)：

- P90 TTFT = 0.4s, 人为定义在prefill阶段90%的请求的TTFT必须达到0.4s (上面那张图黑色虚线部分)
- P90 TPOT = 0.04s, 人为定义在decode阶段90%的请求的TPOT必须达到0.04s (下面那张图的需要部分)

我们先只看图中的蓝线部分：

- 在蓝线部分对应的实验中，我们采用PD合并的架构
- 横轴表示每秒到达这块gpu上的请求数量，记为rps(requests per second)
- 随着rate增加，我们统计每个请求的TTFT和TPOT指标，进而绘制出蓝线。

- 将蓝线和我们设定的SLO（黑虚线）对比发现，如果我们同时达到设定好的TTFT SLO和TPOT SLO，我们的最大rps = 1.6，我们也记这个指标为goodput。

再来看图中的黄线和绿线部分：

- 还是这块gpu，现在我们做两次实验，一次实验只让它处理需要做prefill的请求（黄线），一次实验只让它处理需要做decode的请求（绿线）
- 按照前文介绍的流程，我们得到在这块gpu上只做prefill时，它的goodput=5.6；只做decode时，它的goodput=10。

从这个实验中，我们可以发现，一张卡只做prefill或者只做decode的goodput，都要比它既做prefill又做decode的goodput要高，这也验证了我们前文的猜想：合并式架构可能会同时损害TTFT和TPOT。

假设现在我们不考虑KV cache传输的网络成本，观察上述实验结果可发现：纯decode的goodput是纯prefill的goodput的2倍，所以我们可以估算：如果我拿2张卡纯做prefill，1张卡纯做decode，那么我这三卡构成的架构的goodput就为10，也即单卡goodput = 3.3。这是合并式goodput（值为1.6）的2.1倍！

再换一个角度粗糙地看：分离式架构中，我们三张卡承受起了10 reqs/s的流量；而合并式架构中由于单卡流量承受为1.6 reqs/s，所以我们需要6张卡才能承受起10 reqs/s的流量。因此回到2.1结尾中我们的疑问，乍一看分离式架构似乎更耗卡了，但实际上如果优化得当，它其实是比合并式架构更加省钱的。

三、分离架构下prefill/decode的优化方向

通过上面的分析，我们具象化体会到prefill和decode分离的好处。在本节中，我们再进行一步，详细来看当我们确定可以使用分离架构后，prefill和decode阶段都有哪些优化方向。

3.1 算力与存储的独立优化

- **prefill阶段：拥有计算受限的性质（compute-bound）**，特别是在请求流量较大，用户的prompt也比较长的情况下。prefill阶段算完KV cache并发给decode阶段后，理论上prefill就不再需要这个KV cache了（当然你也可以采用LRU等策略对KV cache的保存做管理，而不是一股脑地清除）。
- **decode阶段：拥有存储受限的性质（memory-bound）**，因为token by token的生成方式，decode阶段要频繁从存储中读取KV Cache，同时也意味着它需要尽可能保存KV cache。

对comput-bound和memory-bound不理解的朋友，可以参见我之前的这篇文章（TODO）。正是由于这两个阶段这两种特性，我们在为其做硬件分配时，也有了更加灵活的空间，而不是只使用一套型号完全一致的硬件。也就是在分离式框架下，计算和存储可以朝着两个独立的方向做优化。

3.2 batching策略的独立优化

我们依然先展示DistServe所作的一个实验结果。

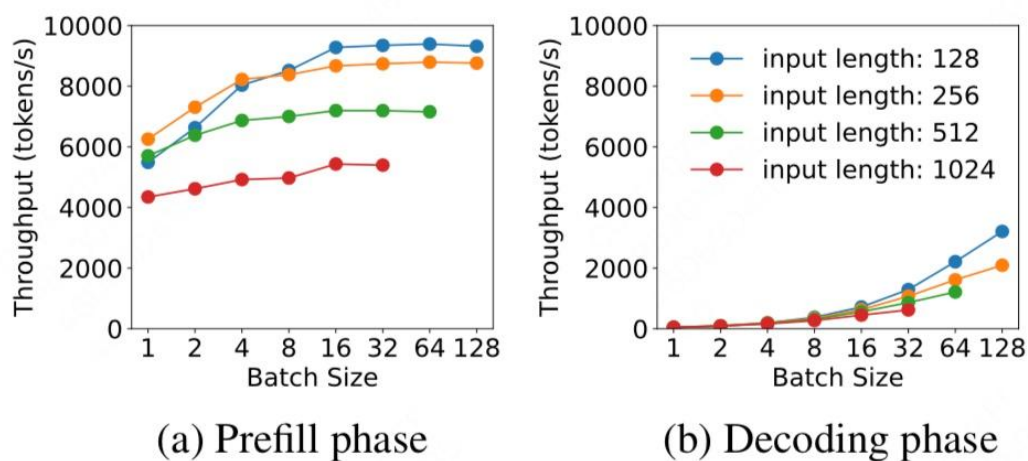


Figure 3: Throughput for two phases with different batch sizes and input lengths when serving an LLM with 13B parameters

这个实验比较了在不同batch size下，prefill和decode阶段的吞吐量情况（tokens/s，即每秒能处理的tokens数量）不同颜色的线条代表不同大小的input_length。

- **prefill阶段：**随着batch size的增加，吞吐量的增长趋势却趋于平缓。这正是因为prefill阶段是compute-bound的，当batch中的总tokens数大于一个阈值的时候，prefill阶段产生compute-bound，使得吞吐量的增长趋势变缓（这里我们不考虑显存问题，因为在该实验中硬件显存足够这些不同batchsize和input_length的数据使用）。
- **decode阶段：**随着batch size的增加，吞吐量的增长趋势越来越显著。这是因为decode阶段是memory-bound的，即相比于计算，读写数据的时间要更多。所以在decode阶段中，如果我们能提升batch size，就能把计算强度提起来，吞吐量就上去了。

通过这个实验，我们不难得出结论：prefill和decode不适合用一样的batching策略，所以当两者分离后，我们可以在batching上继续做优化。

3.3 并行策略的优化

在合并式框架中，prefill和decode共享一个模型，这意味着它们共享一样的dp/tp/pp/....等并行策略。考虑2.3中的实验结果，我们根据实验指标，认为可以设计一个**2个prefill实例（dp=2），1个decode实例（dp=1）的分离系统**。所以我们自然而然想到，如果prefill和decode分离了，那是不是意味着它们的并行策略可以各不相同呢？

由于在前面的定义中，我们认为一个prefill/decode实例包含一个完整的模型副本。所以在本节的并行策略中，我们只考虑tp + pp（即1个实例只通过tp + pp划分）。这样一来，只要对这两个阶段，分别确认好最佳的tp + pp策略，我们就可以用这个策略复制多个prefill/decode实例了（dp维度）

明确了这一点，接下来，我们通过一个有趣的实验 + 数学建模的方式，来看一下prefill和decode在并行策略（tp, pp）上的倾向性。

(1) prefill阶段的并行策略

怎么样可以知道不同并行策略对prefill阶段的影响呢？最简单的方法当然是通过做实验。实验我们肯定还是会做的。但在展示实验结果之前，我们尝试一个有趣的思路：能否拟合一个数学模型，来评估不同并行策略对prefill阶段的TTFT指标的影响？

如果我们能做这样的数学建模，同时用实际实验结果去论证数学模型的有效性，那么之后我们的系统在选择最佳并行策略时，就不需要通过做实验（比如模拟假数据，送入模型实际做fwd）的方式去搜索最佳的并行策略了。我们完全可以用自己的数学模型去估算一个结果，这样岂不是能降低系统在初始化时的耗时？

在DistServe中，在研究不同并行策略对prefill阶段TTFT的影响时，就使用了“[排队论](#)”建立了这个数学模型。不过由于这个模型中一些强假设性（我们马上会讲这点），它并没有用在最终的代码实践中（代码实践换了一种建模方式，我们马上也会谈）。不过因为我觉得这个基于排队论的建模挺有意思的，所以在这节中也会详细来讲。

在下面的讲解中，对于prefill阶段，我们分别对于单卡，2卡pp，2卡tp的情况做基于排队论的建模（没错，这里我们先不谈tp + pp的混合情况，这里我们主要想看对于prefill阶段，它对纯tp和纯pp的倾向性）。

单卡下prefill TTFT

我们假设队列中的请求服从排队论中的M/D/1模式，这个模式的含义如下：

- **M**：请求的到达过程遵从柏松分布。即请求的到达系统是相互独立的（不存在互相干扰）且单位时间内各请求到达系统的概率是等可能的。
- **D**：假设所有请求做prefill的处理时间相同。这个处理时间是指请求被喂给模型一直到模型吐出结果的时间。这就是我们前文说的“强假设”情况。
- **1**：可以理解成只有1张gpu

在这个模式下，单卡下prefill阶段的avg_TTFT可以被建模成：

$$Avg_TTFT = D + \frac{RD^2}{2(1-RD)}. \quad (1)$$

- **D**：即前文所说的单个请求处理时间
- **R**：即遵从柏松分布的请求到达率（每s有多少个请求到达gpu）上。

我知道你现在一定对公式（1）一头雾水，不要急，我们这就来详细解释它。想象一下，对于队列中的某个请求，它的TTFT要怎么计算呢？稍一动脑，我们不难从直觉上给出：

一个请求的TTFT = 这个请求在队列中等待的时间 + 单个请求被处理的时间 = 排在这个请求之前的请求数量

* 单个请求被处理的时间 + 单个请求被处理的时间

这里，单个请求被处理的时间 = D，而根据排队论，我们可以从概率论的角度推算出（推导过程略）排在这个请求之前的请求数量 = 。这样，我们就得到了公式（1）

最后注意，在M/D/1理论的假设下，RD一定是小于1的，如果RD大于1，则意味着这个系统的队列长度会趋向无限（即处理速度比进入速度慢，且进入是不停歇的），那么这时候就不再适合用这个排队理论了。

2-way PP下prefill TTFT

现在，我们假设有两张卡，我们对模型做2-way pp，则此时prefill阶段的TTFT建模如下：

$$Avg_TTFT_{inter} = D_s + \frac{RD_m^2}{2(1 - RD_m)} = D + \frac{RD^2}{4(2 - RD)}. \quad (2)$$

知乎 @猛猿

下标inter表示层间切分，在这篇文章里我们都表示成pp，其余指标基本不变，只是D变了，具体来说有：

。哦？这是什么意思呢？

- **：这个Ds表示这个请求的处理时间**，也还是为这个请求被喂进模型到这个模型吐出相应结果的时间。所以它和D是近似的。
- **：这个Dm就不太相同了**。想象一下排在这个队列中最前面的两个请求。在单卡情况下，当第一个请求进入模型后，第2个请求需要等待D的时间才能被处理。但是在2-way PP的情况下，第1个请求过完第1张卡，第2个请求就可以跟上了，所以第二个请求理论上的等待时间预估为 $\frac{D}{2}$ ，这也就是我们的Dm。**再严谨一点，Dm可以认为是pp过程中最慢的那个stage的处理耗时。**

2-way tp下prefill TTFT

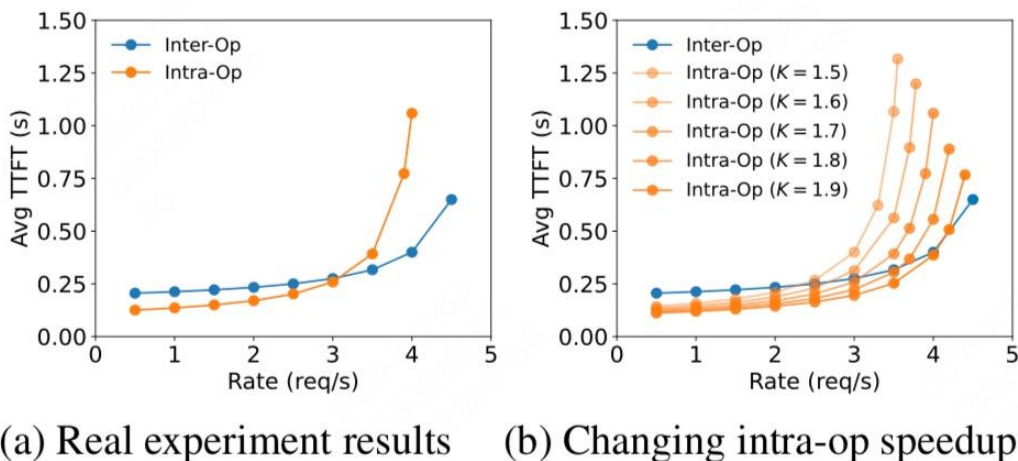
同样假设有2张卡，做2-way tp，则我们有：

$$Avg_TTFT_{intra} = \frac{D}{K} + \frac{RD^2}{2K(K - RD)}. \quad (3)$$

这里的K可理解成是一个折损因子。理论上，如果不考虑因为tp产生的通讯消耗，那么此时在多个gpu上做不同head的并行计算，可能是比单个gpu同时做这些head的并行计算要快的。**但是如果考虑了通讯消耗，那么这个加速时间就有所折损**，我们设 $1 < K < 2$ 。不同的设备拥有不同的折损因子，它对TTFT也是有影响的。

建模 VS 实验结果

建模建完了，这下我们可以用真实的实验结果看看建模拟合效果怎么样了。在DistServe中，确实做了这样的一个实验，同时也提到了建模效果基本拟合的试验曲线的趋势（但是没有单独给出建模效果的趋势图）。我们来看下实验效果：



(a) Real experiment results (b) Changing intra-op speedup

Figure 4: Average TTFT when serving an LLM with 66B parameters using different parallelism on two A100 GPUs @ 猛猿

如图，左侧为真实实验的结果。关于右侧的图，我这里有些迷惑，它似乎是在论证tp模型中取不同折损因子K值时对TTFT的影响（黄线），但是如果是这样，这个曲线应该用建模结果表示的，但它现在却长得和真实实验结果一致。

但是anyway，回到左侧的真实实验结果上，我们不难发现，对于prefill阶段，在rate较小时，更适合用tp；在rate较大时，更适合用pp。也即prefill阶段在不同条件下，对并行方式有倾向性。

(2) decode阶段的并行策略

这里DistServe不再额外建模了，但是它从另外一个角度做了模拟实验的结果：

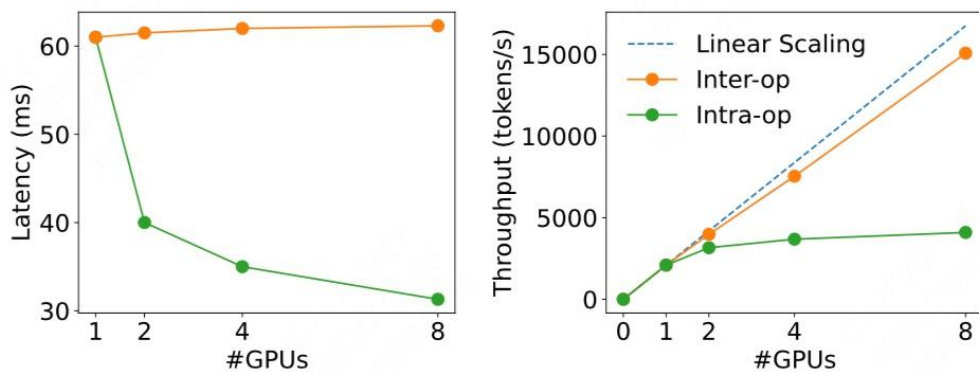


Figure 5: Decoding phase latency and throughput when serving a 13B LLM with batch size = 128 and input length = 256 under different parallel degrees.

知乎 @猛猿

- latency: 模型处理完一个请求的耗时
- throughput: 单位时间内模型能处理的token数量

可以发现，对于decode阶段，随着gpu数量的增加，如果采用pp的方式，能产生更高的吞吐量（理由上面讲过，PP对batch的处理时流水线式的）；但是如果用tp，则能产生更低的延迟（即虽然单位时间内处理的tokens数量有限，但是处理单个请求的时间变快了）。

四、实操：代码实践中的prefill/decode并行策略

在上述的讲解中，我们通过实验的方式，比较直观感受到prefill和decode阶段对于不同并行策略的倾向性。我们还首次尝试通过数学建模的方式，来拟合在不同并行策略下系统的性能曲线（不过正如前文所说，我们初次尝试的这个数学模型具有一些强假设，使得它不适合用在实践中。因此在实践中我们可能要考虑别的建模方式，我们马上来谈着点）。在本节中，我们来具体看DistServe在代码实践中是如何为prefill和decode阶段分配最佳并行策略的。

在开始讲解之前，我们先明确实操中DistServe的整体优化思路：

- （1）对prefill和decode分别做goodput最大化，也就是对他们分别寻找最佳的并行策略（确定一个实例/一个模型副本要怎么分割）
- （2）根据系统中的流量大小确定prefill/decode下一共要部署多少个实例，然后按照（1）中确定的并行方式做实例部署
- 在DistServe中，管这样的策略叫placement。

接下来，我们再来看两种不同的硬件场景：

- 场景1：node间带宽十分高，这样我们可以忽略KV cache的传输时间。我们使用algorithm1来寻找在这样的场景下对单个prefill/decode实例的最佳并行策略。值得说明的是，单node推理也属于该场景

- 场景2: node间的带宽并不高, 不能忽略KV cache的传输时间。这种情况下, 我们尽量把prefill和decode实例对应的模型层分在同一个node中 (也就是prefill和decode的pp维度必须相同)。这种是实践中更普遍的场景。

【本节根据论文伪代码及DistServe开源代码库撰写而成。

<https://github.com/LLMServe/DistServe/tree/main/simdistserve>】

4.1 伪代码：寻找最佳并行策略

Algorithm 1 High Node-Affinity Placement Algorithm

Input: LLM G , #node limit per-instance N , #GPU per-node M , GPU memory capacity C , workload W , traffic rate R .

Output: the placement $best_plm$.

```

 $config_p, config_d \leftarrow \emptyset, \emptyset$ 
for  $intra\_op \in \{1, 2, \dots, M\}$  do
  for  $inter\_op \in \{1, 2, \dots, \frac{N \times M}{intra\_op}\}$  do
    if  $\frac{G.size}{inter\_op \times intra\_op} < C$  then
       $config \leftarrow (inter\_op, intra\_op)$ 
       $\hat{G} \leftarrow parallel(G, config)$ 
       $config.goodput \leftarrow simu\_prefill(\hat{G}, W)$ 
      if  $\frac{config_p.goodput}{config_p.num\_gpus} < \frac{config.goodput}{config.num\_gpus}$  then
         $config_p \leftarrow config$ 
       $config.goodput \leftarrow simu\_decode(\hat{G}, W)$ 
      if  $\frac{config_d.goodput}{config_d.num\_gpus} < \frac{config.goodput}{config.num\_gpus}$  then
         $config_d \leftarrow config$ 
 $n, m \leftarrow \lceil \frac{R}{config_p.goodput} \rceil, \lceil \frac{R}{config_d.goodput} \rceil$ 
 $best\_plm \leftarrow (n, config_p, m, config_d)$ 
return  $best\_plm$ 

```

Algorithm 2 Low Node-Affinity Placement Algorithm

Input: LLM G , #node limit per-instance N , #GPU per-node M , GPU memory capacity C , workload W , traffic rate R .

Output: the placement $best_plm$.

```

 $config^* \leftarrow \emptyset$ 
for  $inter\_op \in \{1, 2, \dots, N\}$  do
   $\mathcal{P} \leftarrow get\_intra\_node\_configs(G, M, C, inter\_op)$ 
  for  $P_p \in \mathcal{P}$  do
    for  $P_d \in \mathcal{P}$  do
      if  $P_p.num\_gpus + P_d.num\_gpus \leq M$  then
         $config \leftarrow (inter\_op, P_p, P_d)$ 
         $\hat{G}_p, \hat{G}_d \leftarrow parallel(G, config)$ 
         $config.goodput \leftarrow simulate(\hat{G}_p, \hat{G}_d, W)$ 
        if  $\frac{config^*.goodput}{config^*.num\_gpus} < \frac{config.goodput}{config.num\_gpus}$  then
           $config^* \leftarrow config$ 
 $n \leftarrow \lceil \frac{R}{config^*.goodput} \rceil$ 
 $best\_plm \leftarrow (n, config^*)$ 
return  $best\_plm$ 

```

知乎 @猛犸

左图对应场景1, 右图对应场景2。

(1) 场景1

- **N:** 表示一个prefill/decode实例最多能占用多少台node
- **M:** 表示一个node最多有多少张卡
- **C:** 集群中GPU的存储总量
- **W:** 系统的负载情况。这个指标和模型类型、input_length等指标密切相关。在3.3节做排队论建模中, 我们曾假设模型处理单个请求的时间是一致的 (这也意味着我们假设所有请求的input_length长度一样), 这在现实中是不可能的。在现实生活中, 你的这个系统可能部署了不同的模型, 而用户对每个模型的使用有倾向性, 同时每个模型中, 用户传入的input_length也是不同的, 所以我们可能很难对系统的负载情况建模。但是我们可以通过模拟实验的方式, 先收集一段时间的用户真实数据, 然后用一个模型计算出用户真实请求服从的分布。接下来我们只需要从这个分布中做采样, 就可以采用出一批数据帮助我们做并行策略的profiling了。我们可以把这批采样出来的数据理解成是系统的负载 W 。真实代码中的 `load_workload` 就在做这件事。
- **R:** traffic rate, 即单位时间内到达这个系统的请求数量。
- **best_plm:** 最佳策略, 等于 $(n, config_p, m, config_d)$
- **config_p:** prefill阶段最佳的并行方式 (每个config_p装一套完整的模型, 我们称为一个prefill实例)

- **config_d: decode阶段最佳的并行方式**（每个config_d装一套完整的模型，我们称为一个decode实例）
- **n**: 理解为prefill阶段的dp值，即要部署多少个prefill实例
- **m**: 理解为decode阶段的dp值，即要部署多少个decode实例

了解了这些指标，看懂算法1就很容易了。整体采用网格搜索的方法，分别对prefill和decode找到满足最大goodput（这个指标表示满足认为设定的SLO值的最大rate值，rate值是单位时间内到达这个实例上的请求数量）。

这里我们只额外介绍一个有趣的函数 `simu_prefill`。

我们以prefill阶段为例，这个函数的输入是当前我们搜索到的一个并行策略，以及模拟出来的负载W。在DistServe的代码实践中，它是用一个“二分搜索”的方法来找到这个goodput值的，我相信很多朋友一定有所疑惑：欸，为什么找goodput值要用二分搜索的办法？

我们先来想一下，当我们决定采用`simu_prefill`时，有哪些数据是明确的：

- **TTFT SLO**：这是你认为设定的指标，即你认为系统要达到这个指标才算合格，因此它是确定的，它和算法无关。
- **并行策略G**
- **负载W**：我们前面说过，这是我们根据历史数据拟合出来的输入数据的特征分布（input_length之类），然后用这个分布进行采样，得到了一批数据。接下来我们就要考虑把这批数据按照什么样的节奏发送到你的prefill实例上。

那这个“节奏”是指什么呢？我们再回想一下2.3实验结果中的那张图（还记得吗，我们就是通过这张图定义了goodput的含义的），**不难理解，这个节奏其实就是指发送rate，也就是goodput的备选值。**

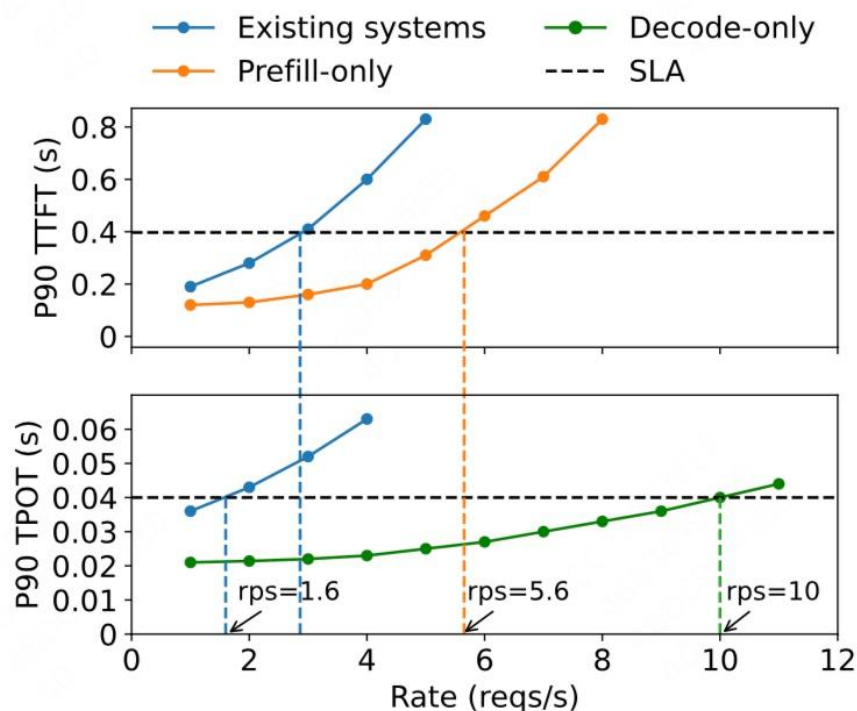


Figure 1: Performance when serving an LLM with 13B parameters under a synthetic workload with input length = 512 and output length = 64 on one NVIDIA 80GB A100. *Upper*: The P90 time-to-first-token (TTFT) latency comparing existing systems vs. a system serving only the prefill phase. *Down*: The P90 time-per-output-token (TPOT) latency comparing existing systems vs. a system serving only the decoding phase.

所以，goodput不是我们通过 `simu_prefill` 函数直接算出来的：

- 它首先是一串备选值（比如 `goodput = 5` 就意味着 `rate` 在 0~5 间都可以）。
- 然后我们以不同的 `goodput`（节奏）发送模拟的数据（负载）到 `prefill` 实例上，按照当前设定的并行策略实际运行一次系统做推理，得到 P90 TTFT 指标。
- 将 P90 TTFT 指标和我们的 P90 TTFT SLO 做对比，如果这个小于 SLO，则意味着可以加大 `rate`，反之同理。
- 那既然现在 `goodput` 是一串备选值，那么我们就可以使用二分法，根据 P90 TTFT 和 SLO 的比较结果，改变我们搜索的上下界，找到最终的 `goodput`（leetcode medium 之魂燃起）

好，现在我们已经理清了 `simu_prefill` 的运作流程，到这一步为止，我们是通过模拟实验的方式找到 `goodput`，进而确定最佳并行策略的。而实验的目的则是获取 `prefill` 阶段的 TTFT 指标。但是，正如前文所说，如果我们能找到一种更确切的建模方式，对 TTFT 进行建模，那么我们就可以拥有函数式的 `simulator`，这样在后续就可以避免每次都跑模拟实验了。再详细一点，我们可以先跑几次实验结果，然后用这个实验结果作为真值，对我们的数学模型中的参数进行拟合。关于这一点，我们放在 4.2 节中详细介绍。

(2) 场景2

理解场景2伪代码的关键就是明白，在场景2下node间的带宽较低，因此KV cache的传输时间不能被忽略。所以我们需要让decode和prefill实例对应的模型层在同一个node里，这也意味着它们的pp维度必须一致。在这个基础上理解2的伪代码就不难啦（脊椎病犯了，实在是敲不动了）。

(3) 并行策略结果

最后，让我们来看下DistServe实操中对于prefill和decode找到的最佳并行策略吧（注意，在175B的模型中，一个实例肯定是要跨机的）

B DistServe Placements in End-to-end Experiments

Table 3 shows the tensor parallelism (TP) and pipeline parallelism (PP) configurations for prefill and decoding instances chosen by DistServe in the end-to-end experiments §6.2.

Model	Dataset	Prefill		Decoding	
		TP	PP	TP	PP
OPT-13B	ShareGPT	2	1	1	1
OPT-66B	ShareGPT	4	1	2	2
OPT-66B	LongBench	4	1	2	2
OPT-66B	HumanEval	4	1	2	2
OPT-175B	ShareGPT	3	3	4	3

Table 3: The parallelism strategies chosen by DistServe in the end-to-end experiments.

知乎 @猿猿

4.2 实操中对TTFT和TPOT指标的建模

A Latency Model for LLM Inference

To accurately simulate the goodput of different placement strategies, we use an analytical model to predict the execution time of the prefill and decoding phases in LLM inference.

In modern LLM serving systems [18, 32, 53], memory-bound operations like Softmax and LayerNorm are usually fused with matrix multiplication kernels for efficiency. Thus the GEMMs dominate the overall latency and our analysis primarily focuses on them.

A.1 Symbol Definition

Here are symbols related to the architecture of the model:

- h : hidden size
- n : number of heads
- s : head size ($h = n \cdot s$)
- m : FFN intermediate size

Note: If tensor parallelism is used, h , n , and m should be divided by the tensor parallelism size.

Below are symbols that characterize the batch to be executed:

- B : batch size
- $l_0, l_1, \dots, l_{B-1}$: input length of each request within the batch
- t : number of tokens in the batch, ($t = \sum_{i=0}^{B-1} l_i$)
- t_2 : squared sum of the input lengths ($t_2 = \sum_{i=0}^{B-1} l_i^2$)
- b : block size in the attention kernel. This parameter is used in FlashAttention [20], a common kernel optimization technique adopted by current LLM serving systems.

A.2 Prefill Phase Latency Modeling

Since the attention operation uses specially optimized kernels, we first discuss the other four matrix multiplications in the prefill phase:

GEMM Name	Shape of M	Shape of N
QKV Linear	(t, h)	$(h, 3h)$
Attn Output	(t, h)	(h, h)
FFN Input	(t, h)	(h, m)
FFN Output	(t, m)	(m, h)

The arithmetic intensity (AI) of these operations is $O(t)$. On NVIDIA A100-80GB GPU, it is compute-bound when AI is over 156. Since t usually can reach several hundred in real cases, all of these operations are compute-bound. Therefore, we can model the latency of these operations according to the total FLOPs:

$$T_1 = C_1 \cdot (4th^2 + 2thm)$$

Next, we discuss the prefill attention operation with FlashAttention [20] optimization. Since the attention only operates among the tokens in the same request, current implementations launch attention kernels for each request in the same batch. For one attention head and a request with

l tokens, the attention kernel needs to perform a total of $2sl + 3sl \cdot (l/b) \approx 3sl \cdot (l/b)$ memory reads and writes, alongside $2sl^2 + sl(l/b) \approx 2sl^2$ FLOPs. So the AI is $2b/3 = 10.677$ (when $b = 16$) or 21.333 (when $b = 32$), indicating that it is a memory-bound operation on A100 GPU. Therefore, the whole attention layer latency (including all requests and all heads) can be modeled as:

$$T_2 = C_2 \cdot n \cdot \sum_{i=0}^{B-1} \frac{3sl_i^2}{b} = C_2 \cdot \frac{3nst_2}{b} = C_2 \cdot \frac{3ht_2}{b}$$

Overall, the latency of the prefill phase can be modeled as:

$$T_{Prefill} = C_1 \cdot (4th^2 + 2thm) + C_2 \cdot \frac{3ht_2}{b} + C_3$$

We use C_3 to quantify other overheads like Python Runtime, system noise, and so on. Then we use profiling and interpolation to figure out the values of C_1 , C_2 , and C_3 .

A.3 Decoding Phase Latency Modeling

Similarly, we first focus on the following GEMMs in the decoding phase:

GEMM Name	Shape of M	Shape of N
QKV Linear	(B, h)	$(h, 3h)$
Attn Output	(B, h)	(h, h)
FFN Input	(B, h)	(h, m)
FFN Output	(B, m)	(m, h)

The AI of these operations is $O(B)$. B is limited by the GPU memory size and stringent latency requirements, so in existing serving scenarios, these operations are memory-bound. The total memory reads and writes is $8Bh + 4h^2 + 2hm + 2Bm$, and since h and m are usually significantly larger than B , we can model the latency as:

$$T_3 = C_4 \cdot (4h^2 + 2hm)$$

As for the decoding attention operation, for one attention head and a request with l generated tokens, it needs to perform $3sl$ memory reads and writes, alongside $2sl$ FLOPs. It is memory-bound, so we can model the latency of decoding attention as:

$$T_4 = C_5 \cdot n \cdot 3s \sum_{i=0}^{B-1} l_i = C_5 \cdot 3ht$$

Summing up, the latency of the decoding phase is:

$$T_{Decoding} = C_4 \cdot (4h^2 + 2hm) + C_5 \cdot 3ht$$

Here we do not introduce the overhead term (like C_3 in the profiling stage) because $4h^2 + 2hm$ is already a constant, and the overhead can be put into C_4 . Similarly, we use profiling and interpolation to figure out the values of C_4 and C_5 .

现在我们来回答4.1中遗留的问题：如果排队论的假设太过理想，那么在实操中，要怎么对TTFT和TPOT指标建模呢？

我们直接放出DistServe论文的appendix部分，这一部分写得非常非常好，短小精炼。但是对于对FLOPs, IO复杂度计算和flash attention不太熟悉的朋友，可能要花一些时间里了解。所以我把这一页都摘出来做说明。

首先我们明确一点，在attention score的计算环节，我们使用了flashattention做特殊优化，因此它和别的环节的TTFT估计还是有一定差别的，所以我们需要把他们分开来说，我们以prefill阶段为例，具体而言：

- 观察A.2中的table，它是去除attention score计算后剩余的GEMM矩阵。shape of M/N分别表示这个GEMM计算的两个矩阵的维度。例如对于QKV linear，它指代的就是我们的输入数据通过Wq/Wk/Wv矩阵产出QKV值的过程。只是这里(t, h)表示我们把所有batch的数据拼接成一条，(h, 3h)表示我们把Wq/Wk/Wv矩阵拼成一个。这个table的其余列同理可推
- 根据这个table列举的GEMM计算，我们可得总FLOPs（对这个计算方式不明确的朋友，可以参看3.1节中给的链接文章），这样就得到T1计算公式中括号里的那一坨
- 由于table中的GEMM计算都是compute-bound的，因此在考虑TTFT时我们可以忽略读取数据的时间。这样，我们设因子C1表示计算单个FLOPs的耗时，那么table中的TTFT消耗就是T1所示。C1则是我们要拟合的模型参数。
- 观察T2，由于attention score的计算是memory-bound的（尽管我们已经用flashattention做过了优化），所以我们不关心它的FLOPs，而关心它的IO复杂度（对这个计算有困扰的朋友依然可以参考3.1链接文章）。然后在这个基础上我们再设因子C2，表示单次读取的耗时。
- 综合T1和T2，建模出最终的prefill TTFT模型。对decode阶段也是同理，不再赘述。

分离式推理架构2，模糊分离与合并边界的chunked-prefills

在[分离式推理架构1](#)中，我们以DistServe为例，解释了“为何要使用分离式推理架构”：分离式推理架构可以解耦prefill (compute-bound) 和decode (memory-bound) 过程，使得不管是在硬件分配还是在并行策略上，这两者都能朝着独立的方向优化，同时改进TTFT和TPOT，而无需再像合并式推理架构那样，总是在这两者之间做trade off。

但是，读完这篇文章，你可能会这样的疑惑：如果我能采取一种方法，使得处于prefill阶段的请求和处于decode阶段的请求能组成一个batch同时计算，而在组建这样的batch的过程中，我又充分考虑了最大化GPU计算单元利用率、最小化IO读写次数（简而言之，怎么能榨干一块gpu我就怎么来）。那么这时，我是不是在不解耦的情况下，同样也能同时保全TTFT和TPOT呢？

那么在这篇文章中，我们就来看看遵从这个思路设计的推理架构：[Sarathi-Serve](#)，以及它背后的核心技术chunked-prefills（切块式prefill）和stall-free schedules（无停滞式调度策略）。虽然本文是讲Sarathi-Serve，但是为了更好理清其设计思路（它也是在借鉴了其余架构的基础上改良而来），本文也会涉及对其余架构的核心技术讲解，并依次回答两个重要问题：

- 为什么有了[selective batching](#)还可能还需要chunked-prefills
- 为什么有了chunked-prefills还可能还需要分离式架构

一、传统batching方式

1.1 整体流程

我们来看早期一个传统的batching方式的例子（例如[FasterTransformer](#)的实现，图片来自[Orca](#)论文）：

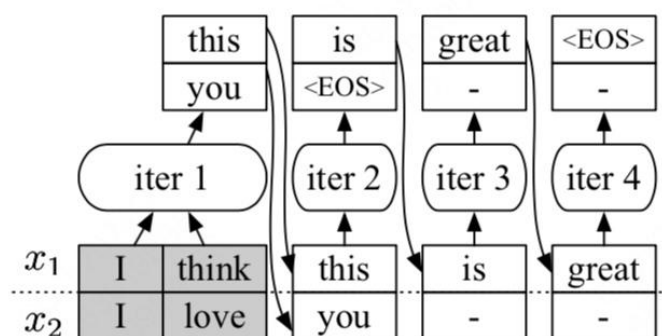


Figure 3: An illustration for a case where the requests have the same input length but some requests finish earlier than others. Shaded tokens represent input tokens. “-” denotes inputs and outputs of extra computation imposed by the scheduling

知乎 @猛猿

在这个例子中，我们的batch_size = 2，分别装着长度相等的x1和x2序列（长度不相等时，可以采用诸如左侧padding等方法）。

- 我们把（左padding过后）长度相等的序列送入模型做prefill，产出第一个token。整个prefill的过程，被称为1次iteration（中文可以理解成一次迭代，或者1个推理阶段）。
- 接下来我们对这两个序列做decode。可以发现1次迭代后，x2已经推理完毕，x1依然还在做推理
- 由于在传统batching方法中，整个batching中的序列是一起行动的，所以尽管x2已经做完推理了，它还是没有办法被“释放”。“释放”的含义是：x2所占据的资源（例如KV cache等）不能被释放。
- 接下来，x1又做了两次迭代。这下x1也完成推理了。然后整个batch中的数据才可以被真正“释放”。
- 当这一个batch推理完毕后。其余请求才能继续组成新batch，做下一轮推理。

正是由于在传统batching中，需要所有的request一起行动，因此和传统batching配套的调度方式，又被称为request-level schedules

1.2 传统batching方式的缺陷

由1.1的整体流程，我们可以直观看出传统batching方式的缺点：

1. 以牺牲TTFT的方式保全TBT（Time Between Tokens，可以理解成和TPOT是等价的）。由于整个batch一起行动，所以在这个batch做推理的过程中，不能接受新的请求，导致prefill的过程停滞了(stall)。所以尽管它一气呵成完成了现有数据的decode过程，它却增加了新请求们在队列中等待被处理的时间。
2. 以牺牲吞吐（throughput）的方式降低延迟（latency）。由于不能接受新请求，吞吐量下降了，但是由于不间断地做decode，对decode来说延迟降低了。
3. 增加了流水线并行中的气泡。

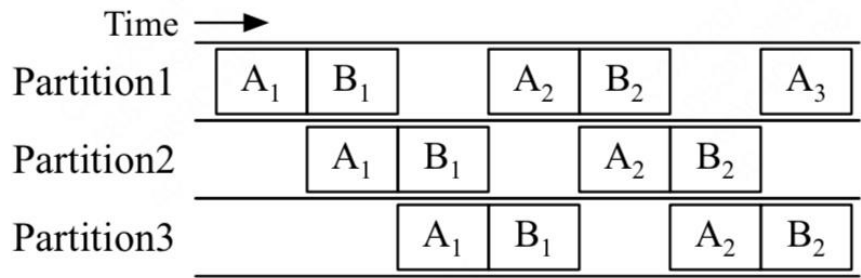
我们对第3点做一些更详细的说明。

在大模型推理中，当模型尺寸过大时，我们需要把它切割到多张卡上，常用的并行方式有pp和tp（这里我们不谈dp，因为确认好tp和pp后，dp维度只是做模型副本拷贝而已）。**一般来说，在做推理时，我们希望用一个较大的batch，这样一来我们可以最大化利用gpu的计算单元，二来也减少从显存读取数据到cache的次数**（比如同样是从显存中读取模型权重，如果你分成很多小batch，你就要读取多次。当你合成大batch时，你只用读取1次，大家共享就可以了）。

- **当我们使用tp时，我们是对模型做层内切割**，这样一块卡上维护的模型权重占的显存就少了，我们就有空间组织更大的batch了。但是由于tp在前向过程中涉及到2次allreduce，所以它对不同gpu间的通讯性能要求更高。**因此一般是在单机内，或者在有更好带宽的集群的情况下，我们会倾向于使用tp。**
- **当我们使用pp时，我们是对模型做层间切割**，一块卡上维护的还是完整的层，虽然此时可能batch无法像tp那样打得比较大，但是pp间只涉及层间activation的通讯，对带宽要求更小。**所以很多商用的架构都会使用pp作为推理的并行方式。**

那么如果使用pp做推理，有一个优化点肯定是避不开的：**减小pp的bubble，也就是减少gpu的空闲时间。**

我们来看传统batching方式下的pp bubble情况，如下图（图片来自Orca论文）：



(b) FasterTransformer execution pipeline.

Figure 8: Comparison of the use of pipeline parallelism in ORCA and FasterTransformer where X_i is the i -th iteration of request X .

知乎 @猛猿

其中，batch_size = 2，它装了A和B两个序列，下标表示序列正在进行第几个迭代。我们假设A和B此时都处于decode阶段。partition1 ~ 3可以理解成是3张gpu，上面维护着模型的不同层。

由于decode阶段是token by token的，所以A和B必须在第1次迭代产出一个token后，才能做第2次迭代。这就造成了每块gpu上的bubble（空闲时间）。

看见传统batching方式的这3个缺陷，此时的你一定觉得很可惜，因为：

- 已经做完推理的请求，为什么还要占据着资源呢？把位置让给新的请求，让新请求做prefill，旧请求继续做decode，那不是更好吗？
- 在使用pp的前提下，我在那些气泡处，塞入新请求做prefill或者decode，不就既能把那些气泡填满，又不影响当前请求做推理吗？

所以，这一切都指向了两个迫切需要被改进的方向：

- 更改request-level的限制，让新请求和旧请求能接连不断组成新的batch（Orca [iteration-level schedule](#)）
- 让prefill和decode能在一个batch中一起做（Orca selective batching）

二、Orca: Selective Batching

2.1 Iteration-Level Schedule

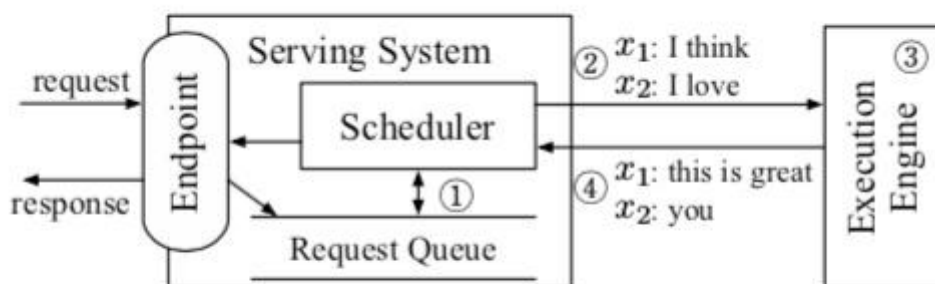


Figure 2: Overall workflow of serving a generative language model with existing systems.

知乎 @猛猿

再复习一下：传统推理架构的调度流程如上图（图片来自Orca论文）。调度器（Scheduler）每次从请求队列中组织一个新的batch（如图中的x1和x2），然后与执行引擎（Execution Engine）交互做推理，等engine把这个batch的数据都做完推理并且返回给用户后，调度器才会继续从请求队列中组织新的batch。由于batch中的所有请求必须一起行动，我们管这种调度策略叫**Request-Level Schedule**。

而我们的目标是：及时检测出推理完毕的请求，将其从batch中移出，好腾出位置给新的请求。那怎么实现这点呢？还记得我们在1.1中给出的那张推理流程示意图吗？在那张图里，我们管请求做完prefill产出第一个token的过程叫1次iteration，请求每做一次decode也被称为1次iteration。所以，对于一个batch内的数据，如果我是按iteration维度调度的，也就是一个batch中的所有请求每做完1次iteration，scheduler就和engine交互一次，去检查batch中是否有做完推理的请求，以此决定是否要更新batch，这样不就能达到我们的目的吗？我们管这样的调度策略叫**Iteration-Level Schedule**，整体流程可用下图表示（图片来自anyscale blog: <https://www.anyscale.com/blog/continuous-batching-llm-inference>）

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3					
S_4	S_4	S_4					

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

Completing seven sequences using continuous batching. Left shows the batch after a single iteration, right shows the batch after several iterations. Once a sequence emits an end-of-sequence token, we insert a new sequence in its place (i.e. sequences S_5 , S_6 , and S_7). This achieves higher GPU utilization since the GPU does not wait for all sequences to complete before starting a new one.

知乎 @猛猿

这里，我们先不要管如何使用特殊的方法让这个batch中的数据能同时做推理（我们马上在下文讲解），只着重关注调度流程。这个batch中原始有4个序列 $s_1 \sim s_4$ ，黄色表示prefill tokens，蓝色表示decode tokens。左图展示了这4个序列刚做完prefill的过程。在此之后序列进入decode阶段，每生成1个token，scheduler就和engine做交互，即时检查序列的完成情况。在右图中， s_3 最先做完推理。此时scheduler检测到了这点，就把 s_3 从batch中移除，再从队列里塞入新请求 s_5 组成新batch继续做推理。 $s_6 \sim s_7$ 的推理过程同理可推。

2.2 Selective Batching

了解了iteration-level schedule后，现在我们要来看一个大家都非常好奇的问题：**同一个batch中，那些形态、计算方式各异请求，要如何同时做推理？**

举例来说：

- prefill过程是长序列并行计算的，decode过程是token by token的
- prefill过程不需要读取KV cache，decode过程需要读取KV cache
- 对于prefill，各个请求的prompt长度是不一致的
- 对于decode，不同请求的decode token的index不一样，意味着它们计算attention的mask矩阵也不一样。

诸如此类，真是令人头大。

而解决这些问题的一个好思路是：**尽量找到这些请求计算时的共同之处，使得计算能最大化合并。对于有差异的部分再单独处理。**这样说你可能觉得比较抽象，不要紧，我们先以一个transformer decode block为例，回顾一下序列要经过哪些计算，然后我们再慢慢讲解合并batch计算的细节。

(1) Decoder block中的各种计算类型

(下图来自sarathi论文)

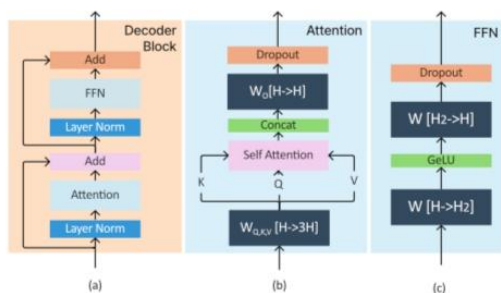


Figure 2: High-level architecture of a decoder block.

Operation	Shapes of tensors		
	Input(s)	Weight(s)	Output(s)
<i>preproj</i>	$[B, L, H]$	$[H, H]$	$[B, L, H]$
<i>attn</i>	$[B, L, H]$	$-$	$[B, L, H]$
<i>postproj</i>	$[B, L, H]$	$[H, H]$	$[B, L, H]$
<i>ffn_in1</i>	$[B, L, H]$	$[H, H_2]$	$[B, L, H_2]$
<i>ffn_in2</i>	$[B, L, H_2]$	$[H_2, H]$	$[B, L, H]$

Table 1: Shapes of the input, weight, and output tensors in a transformer decoder block. B, L and H denote batch size, embedding (aka hidden) size and sequence length ($L=1$ during decode, except for attention).

知乎 @猛猿

specifically that of the decode phase is limited by the max-

- **preproj**：即序列经过 矩阵产出 的过程。观察table1中给出的input和weights权重，可以发现重要的两点：
 - preproj计算时不需要从显存读取模型权重。
 - preproj计算时和input序列长度无关（只是在hidden_size维度上做线性转换）
- **attn**：利用计算出的 计算attention分数的过程，可以发现：
 - attention分数计算时不需要从显存读取模型权重，你只需要利用算好的QKV即可
 - atttention分数计算时依赖mask矩阵，而不同序列的mask矩阵是不同的
- **postproj**：使用 权重矩阵，对经过attention计算后的序列做映射，它的两个特性和preproj一致。
- **FFN1与FFN2**：道理同preproj/postproj，不再赘述。

我们把上面的介绍稍作提炼，得到如下重要信息：

- **preproj/postproj/FFN1/FFN2**：做这些计算时，需要从显存读取模型权重，且这些计算和input序列长度无关。
- **attn**：做attention分数计算时，不需要从显存读取模型权重，且不同序列的mask矩阵不同。

(2) Selective Batching的计算细节

- **preproj/postproj/FFN1/FFN2** 的计算和序列长度无关，这意味着你可以把一个batch中所有的tokens都展平成一行进行计算（维护好各自的位置向量就好）。而这些计算都要读取模型权重，这意味着我们可以尽量增大batch size，使得一次读取能造福更多request，以此减少IO次数。
- **attn** 的计算受各个序列的差异性影响（例如mask矩阵、是否需要读取KV cache），所以需要将序列拆分开独立处理，也即batch维度是重要的（cuBLAS batch matrix multiplication）。而由于attn部分本身不涉及到权重读取，因此你把序列拆分开处理，也不会在这一方面上带来额外的IO开销。

整体流程如下（图片来自Orca论文）：

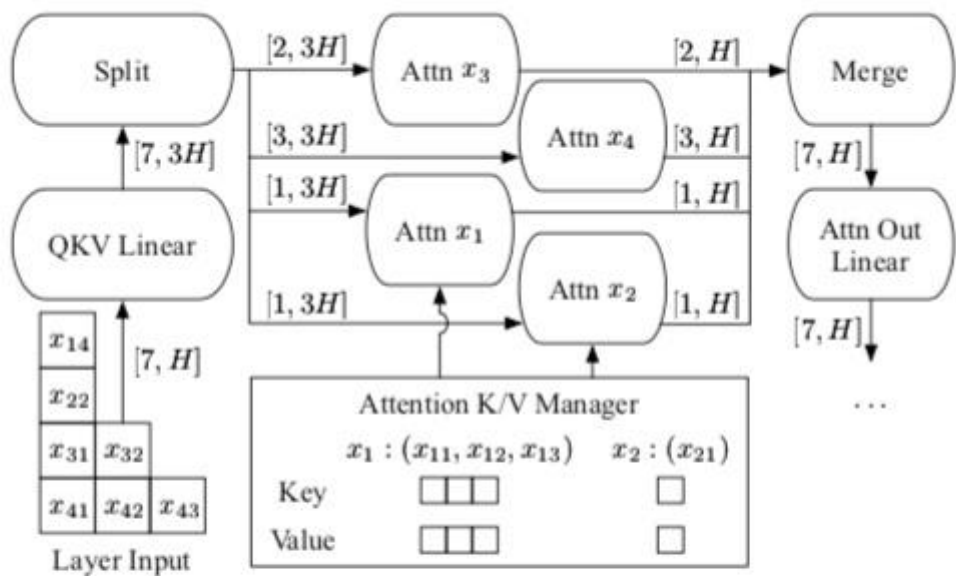


Figure 5: An illustration of ORCA execution engine running a Transformer layer on a batch of requests with selective batching. We only depict the QKV Linear, Attention, and Attention Out Linear operations for simplicity.

知乎 @猛猿

在图中，序列x1和x2正在decode阶段（因此需要KV cache Manager帮它们取出KV cache），序列x3和x4正在prefill阶段，它们被组成了一个batch。在非attention的部分，batch中的7个tokens被拉平成一行进行计算（忽略了batch维度），等实际计算attention时，再split开。计算完后再拉平。

三、Sarathi-Serve: chunked-prefills

我们来小结一下目前为止的内容：

- 我们以分离式架构为引子，讨论了解耦prefill和decode过程带来的好处：能独立优化TTFT和TPOT/TBT，同时提升吞吐和降低延迟。
- 基于此，我们又产生了疑问：如果不采用解耦的方式，只是修改传统的batching里非prefill即decode的方法，在最大化榨干一块gpu的前提下，让prefill和decode能同时放在一个batch里做推理，是不是也能达到一样的效果？
- 为了解答这个问题，我们先回顾了以FasterTransformer为代表的早期batching方法：在推理的每个时刻，batch中的序列总是一起做prefill，或一起做decode。
- 接下来，我们介绍了Orca是如何能让各种请求（prefill+decode，长度不同的prefill，index不同的decode等）混合在一个batch里做同时做推理的。

关于混合batch对性能带来的提升，大家可以去看Orca论文中的实验部分（以FasterTransformer等更早期的推理架构为baseline），[这里就不赘述了](#)。我们来看一个更有趣的问题：**为什么混合batch可以带来性能上的提升？**

3.1 为什么混合batch可以带来性能上的提升

我们来看sarathi-serve做的一个实验（图片来自sarathi-serve论文）

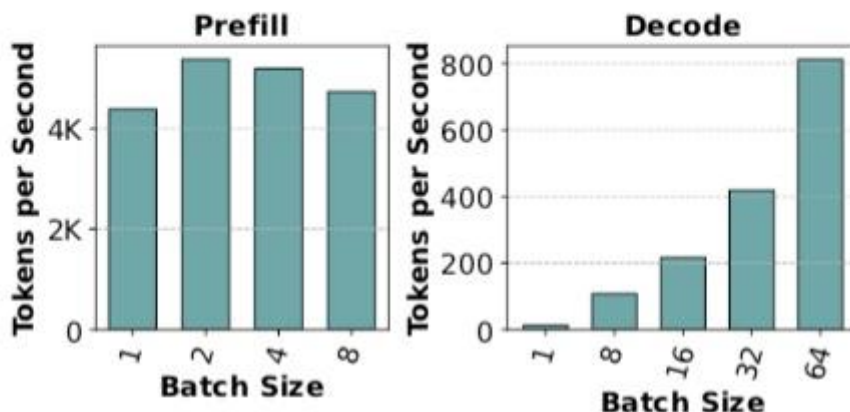


Figure 3: Throughput of the prefill and decode phases with different batch sizes for Mistral-7B running on a single A100 GPU. We use prompt length of 1024 for both prefill and decode experiments. Note that different y-axis, showing prefills are much more efficient than decode. Further, note that *batching boosts decode throughput almost linearly but has a marginal effect on prefill throughput.*

知乎 @猛猿

左右两图分别刻画了在不同的batch size下，prefill和decode阶段的吞吐量（tokens per second，每秒能处理的tokens数量）。

- 观察到，**对于prefill阶段来说，提升batch size时，吞吐量的有增长但不太显著**。甚至当batch size更高时（比如从4~8），还发生了吞吐量的下降。**这是因为prefill阶段是compute-bound的**，也即相比于读数时间，它消耗在计算上的时间更大（由于数据是可以边读边算的，所以我们可以大致认为总时间）。prefill阶段读取数据（例如从显存读取模型权重）的时间成本是固定的，但是计算时间却会随着batch中tokens的数量而增长，因此当gpu的计算单元还没有被打满时，吞吐量还可以上去；被打满时就会下降了。
- **对于decode阶段来说，提升batch size时，吞吐量增长的线性趋势非常明显**。这是因为**decode是memory-bound的**，也就是它花在读数上的时间更大（回想一下，当你用一个token做decode时，你其实要做的新计算很少，大部分时间你都花在读取KV cache和模型权重上）。decode阶段的算力严重打不满，所以当你增大batch size时，你不仅能多利用算力，也能把多次读取合并成一

次读取，吞吐量自然就上升显著了。但是你也不能无止尽地增加batch size，因为gpu的存储是有限的，decode还要读取前面那一长串的KV cache呢。

既然decode和prefill阶段都需要读一些固定的数据（比如模型权重），且decode阶段的算力没有打满，那我们把他们组装在一起，让他们互相搭便车，肯定能取得更好的效果，也即：

- **prefill搭上decode的便车，能用上decode阶段被浪费的算力。**
- **decode搭上prefill的便车，合并数据的读取次数，做到1次读取，大家共享。**

3.2 为什么有了selective batching，还需要chunked-prefills

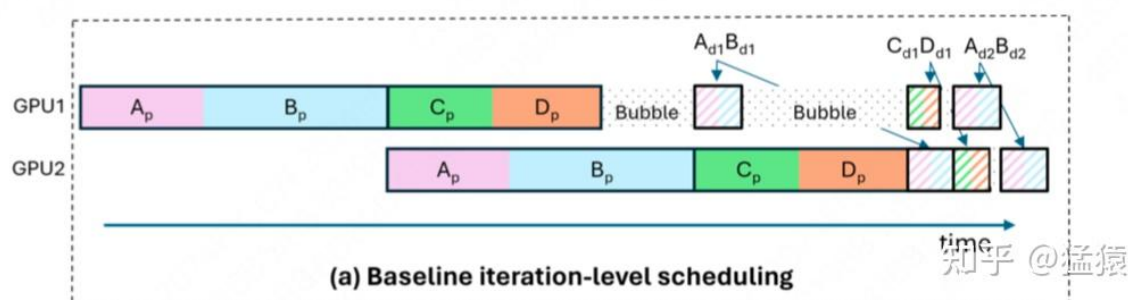
在3.1中，我们介绍了prefill和decode组成混合batch对性能提升的好处：乍一看，既不耽误做prefill（TTFT），也不耽误做decode（TPOT/TBT）。那么目前为止，Orca应该做得挺好了哇，那这个Sarathi-Serve的chunked-prefills，是干什么的呢？

当你回顾Orca组装batching的过程时，你可能会发现这个过程比较随机：一个batch中做prefill和做decode的请求有多少条是不确定的，只是大体按照先来后到的原则做动态组装。这就造成了一些问题：

- 如果一个batch中做prefill的请求非常多，或者做prefill的请求非常长，那么prefill tokens会占据大量计算资源，使得整个batch变成**compute-bound**。
- 如果一个batch中做decode的请求非常多（比如当所有的请求都没做完推理时，或者请求队列中没有新序列可以调度时），这个batch就可能变成**memory-bound**的。
- 随机的batch同样可能产生pp并行气泡。

哦咦，熟悉的感觉，我们再来看看第三点，还是关于pp并行气泡的问题。

我们知道相比于FasterTransformer，Orca已经能在一定程度上改善pp气泡问题了，但是由于其batch组装的随机性，它仍然可能导致气泡问题，我们以下图为例（图片来自Sarathi论文）：



ABCD表示4个队列，下标p表示prefill阶段，di表示decode的第i个阶段。在采用micro-batch的前提下（也是减少pp气泡的一种办法），micro-batch size = 2，AB组成一个小batch，CD组成一个小batch。注意到这两个batch虽然size一致，但tokens数量更不一致。

观察到图中一共有3种类型的bubble：

- **PB1**：因为micro-batches中prefill序列长度不一致而产生的bubble
- **PB2**：因为prefill和decode阶段计算时间的差异而产生的bubble

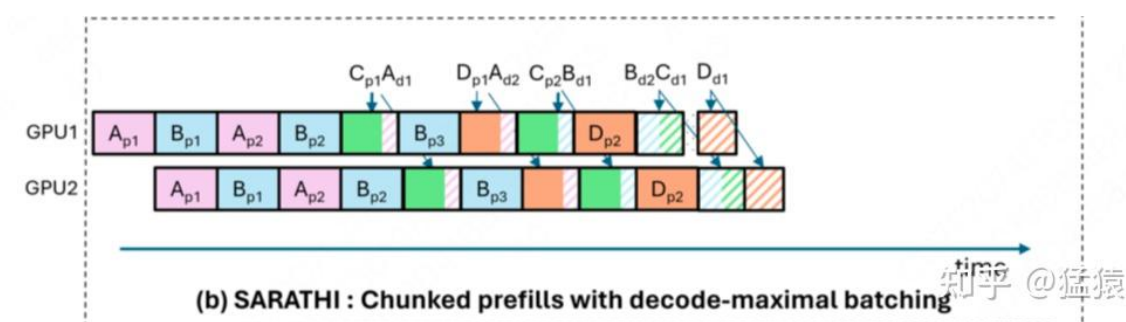
- **PB3**: 不同micro-batch的decode差异性而产生的bubble, 这是因为不同micro-batch在做decode时, 要读取的KV cache的长度不一致, 这也导致了在读取数据上所花费的时间不一致

基于Orca selective batching的这些缺陷, 我们不禁想: **如果我们在保持selective batching这种混合机制的情况下, 根据gpu资源的上限 (FLOPS/MemBandwidth), 找到一个最大batch size, 即定义好一个batch内最多能处理的tokens数量, 然后在每个batch内, 在按照一定比例去分配做prefill的tokens和做decode的tokens, 不就既能解决pp并行中的气泡问题, 又能让这个batch得到性能最大化吗?**

而在这种解决办法下, 一个请求用于做prefill的序列必定是要被拆开的, 所以我们就管这种方法为:
chunked-prefills

3.3 chunked-prefills运作流程

基于pp的chunked-prefills运作流程如下 (图片来自Sarathi论文):



- 首先, 我们通过3.2中的思路, **从我们所使用的gpu性能出发, 确定每个batch中最多能处理的tokens数量** (可以通过profiling做模拟实验得到)。
- 然后, 我们在各个batch中进一步确定prefill tokens和decode tokens的比例。确认的原则被称为**“decode-maximal batching”**: 即优先往batch中添加需要做decode的序列, 直到添加不动为止 (即我们预留给decode的KV cache空间已经不足了, 无法存放新的KV cache了)。然后我们再根据这个batch中剩余的tokens预算, 对需要做prefill的序列做chunk切割, 把对应的prefill tokens添加进batch中

最后, Sarathi-Serve依然采用的是iteration-level schedules, 即推理的每一步后, scheduler都会重新组建batch。

【我们会在本章最后一节解读Sarathi-Serve调度器策略的源码, 给大家展示更多上述流程的细节, 这里大家只需要大致了解chunked-prefills的运作流程即可】

chunked-prefills的额外开销

看完了运作流程, 你肯定有这样的疑惑: **原来一条序列做prefill时, 我是一起计算的。现在我把它拆成了多个chunk, 那么每个chunk去计算时, 肯定要去读前一个chunk的KV cache (如下图), 那不就增加了IO复杂度了吗? 这会影响prefill计算的性能吗?**

	k0	k1	k2	k3
q0	1	-	-	-
q1	1	1	-	-
q2	1	1	1	-
q3	1	1	1	1
attention mask during first chunk prefill				

	k0	k1	k2	k3	k4	k5	k6	k7
q4	1	1	1	1	1	-	-	-
q5	1	1	1	1	1	1	-	-
q6	1	1	1	1	1	1	1	-
q7	1	1	1	1	1	1	1	1
attention mask during second chunk prefill								

	k0	k1	k2	k3	k4	k5	k6	k7	k8	k9	k10	k11
q8	1	1	1	1	1	1	1	1	1	-	-	-
q9	1	1	1	1	1	1	1	1	1	1	-	-
q10	1	1	1	1	1	1	1	1	1	1	1	-
q11	1	1	1	1	1	1	1	1	1	1	1	1
attention mask during third chunk prefill												

Figure 6: Example of how attention mask is set across different chunk prefill iterations in SARATHI (q and k represent “query” and “key” tokens, respectively). The attention mask for v (“values”) is set similarly.

知乎 @猛猿

这个读取KV cache的额外开销肯定是有，但它对prefill的影响大吗？基于此，Sarathi-Serve的作者们做了两个实验。

第一个实验：证明prefill阶段是强compute-bound特性，以及计算attention的时间在总计算时长里占比不高。

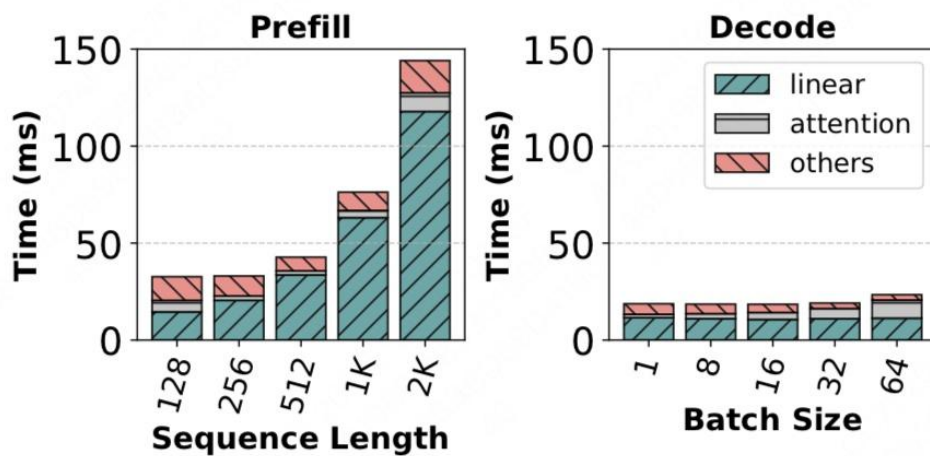


Figure 4: Prefill and decode time with different input sizes for Mistral-7B running on single A100 GPU. Linear layers contribute to the majority of runtime in both prefill and decode phases. Due to the low arithmetic intensity in decode batches, the cost of linear operation for 1 decode token is nearly same as 128 prefill tokens.

知乎 @猛猿

我们知道KV cache仅用在attention的计算中，所以这里作者把时间消耗拆成了attention和非attention (linear + others) 的部分。可以发现：

- 对于prefill的部分，不管prefill tokens数量如何，attention部分的计算时间在总时长里占比并不高。
- 对于prefill部分，随着seq_length的变长，tokens的处理时间也变长。但是在128~512的长度内，tokens的处理时间增长不显著。这是因为在这个范围内，gpu的算力还没有打满。在这之后进入强compute-bound区域，此时读取数据的时间对prefill来说影响更小。

第二个实验：直接比较chunked-prefills和正常prefill下的延迟

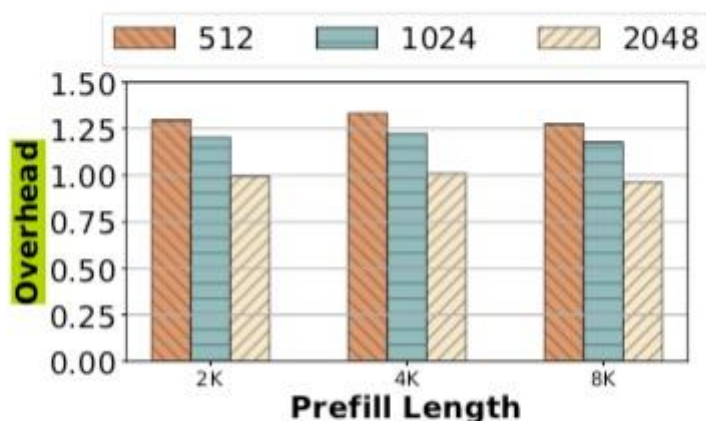


Figure 14: **Overhead** of *chunked-prefills* in prefill computation for Yi-34B (TP-2) normalized to the cost of no-chunking, shown for various prompt lengths using chunk lengths of 512, 1024 and 2048.

知乎 @猿猿

这里以正常prefill为baseline（设其overhead = 1，即没有额外开销），比较不同chunk size下的额外开销。不出意外，prefill chunk分得越细（例如512），开销越大，但是总体来说，开销增长都控制在1.25倍内。稍微影响到TTFT，但是考虑到它对TBT/TPOT的更多提升（可以参见论文别的实验，这里不再写出），这样的开销还是可以接受的。

3.4 stall-free schedules

在Sarathi-Serve的设计思想下，**无论是prefill过程还是decode过程，都不会产生停滞（stall）**。以Sarathi-Serve作者的观点来看：在其余的推理架构中（比如vllm, Orca, FasterTransformer），他们都或多或少存在停滞一方以保存另一方的策略，我们来看一个整体流程图（图片来自Sarathi-Serve论文）：

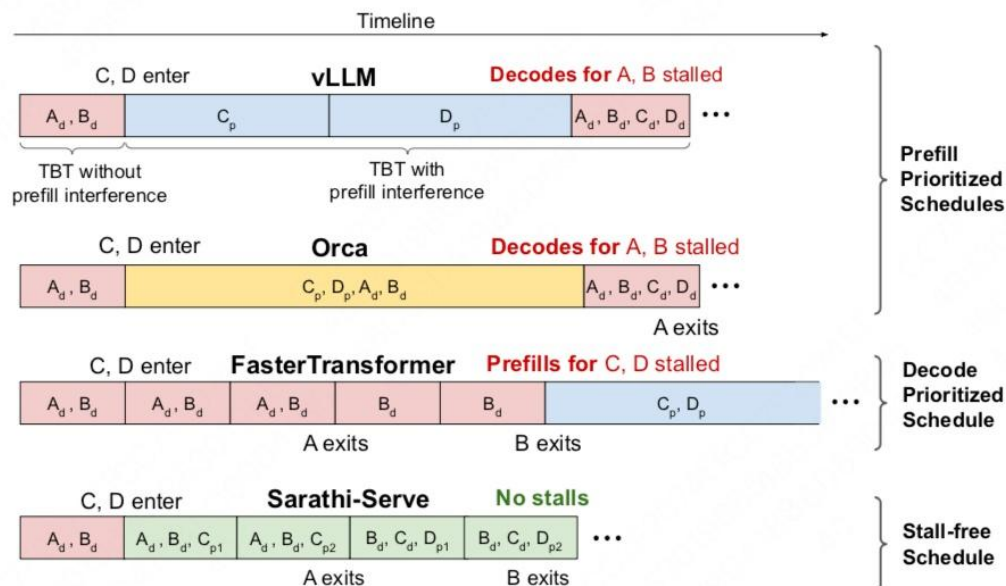


Figure 7: A generation stall occurs when one or more prefills are scheduled in between consecutive decode iterations of a request. A, B, C and D represent different requests. Subscript d represents a decode iteration, p represents a full prefill and $p0, p1$ represent two chunked prefills of a given prompt. vLLM induces generation stalls by scheduling as many prefills as possible before resuming ongoing decodes. **Despite supporting hybrid batches, Orca cannot mitigate generation stalls because the execution time of batches containing long prompts remains high.** FasterTransformer is free of generation stalls as it finishes all ongoing decodes before scheduling a new prefill but compromises on throughput due to low decode batch size. In contrast, Sarathi-Serve generates a schedule that eliminates generation stalls yet delivers high throughput.

假设最开始有A、B两个序列，他们都处在decode阶段。从上帝视角来看，A和B分别要经过2次、4次decode迭代才能完成推理。

- 对于这4个框架，A和B首先进入第1次decode迭代（图中第一个红色方块）。到这一步为止这4个框架没有什么差异。
- 当A和B完成第一次decode迭代后。新来了请求C和D。
- 对vllm，我们在之前的源码解读系列说过，它是prefill优先的，所以它会先处理C和D，这就使得decode暂停了（stall）。这其实是在保吞吐弃延迟（使得TBT增加了）
- 对Orca，它在硬件资源允许的情况下，是可以让CD做prefill，AB继续做decode的（黄色部分）。但是由于decode和prefill的完整序列绑定，也使得整个decode的计算时间变长了（特别是在CD是长序列的情况下）。所以这其实也算是一种decode暂停
- 对于FT，它是保延迟弃吞吐的。这使得prefill暂停了。

- 对于sarithi-serve，它和orca一样，也是允许decode和prefill一起做的，但是它通过合理控制每个batch中prefill tokens的数量，使得decode阶段几乎没有延迟（把sarithi的绿色块和FT的红色块相比，可以发现绿色块只长了一点）。这样即保了延迟，又保了吞吐。

3.5 Sarathi-Serve调度流程源码解析

由于Sarathi-Serve论文中的调度流程伪代码，和实际的源码实现存在一定的差异。所以我这里直接根据源码来分析使用chunked-prefills方法时的调度流程（给出了非常详细的注释，大家可以关注下~）：

```
class SarathiScheduler(BaseScheduler):

    def __init__(
        self,
        model_config: ModelConfig,
        scheduler_config: SarathiSchedulerConfig,
        cache_config: CacheConfig,
    ) -> None:
        super().__init__(model_config, scheduler_config, cache_config)

        # =====
        # 【固定chunk_size策略】
        # 人为定好的chunk_size。如果你不想动态变更chunk_size大小，你可以固定使用这个。
        # 我们可以通过profiling等方式，在调度开始前确定好能够
        # saturate gpu computation的最大chunk_size
        # （注：在代码中，chunksizes不是指prefill的chunksizes，是指每次
        # 调度中，整个batch的tokens数量，也包括要做decode的tokens数）
        # =====
        self.chunk_size = self.scheduler_config.chunk_size

        # =====
        # 【动态chunk_size策略】
        # 使用动态变化的chunk_size
        # （随着调度次数增加，历史累积的要做的decode的序列可能会变多，以及
        # 可能会进来更多的新请求。假设某个序列的prompt特别长，那么它就会持续占据着计算
        # 资源，影响到别的请求。所以对于这样的prompt，我们可以在迭代中逐渐减小它的preill
        # tokens数量）
        #
        # 为了执行这个chunk_size动态变更的策略，我们需要如下4个参数：
        # 【low_chunk_size】：人为设定的最小chunk_size
        # 【high_chunk_size】：人为设定的最大chunk_size
        # 【chunk_schedule_stages】：用于刻画调度阶段数。例如该值若等于5，则说明随着
        # 调度次数的增加，我们希望能有5种逐步递减的chunk_size可以选择
        # 【chunk_schedule_max_tokens】：这个变量比较难说明，我们直接看它怎么用。
        # 事实上，在源码中真正有意义的变量是_tokens_per_stage
        # （=chunk_schedule_max_tokens/chunk_schedule_stages）
        # 你可以理解成：对于一个正在做prefill的长序列，我们它的prefill tokens数量
        # 随着迭代阶段（stage）的增加而递减。我们设其做prefill时，每处理_tokens_per_stage
        # 个tokens就算完成了1个stage，然后就要递减一次prefill tokens。简而言之，这些
        # 参数的作用是帮助我们确定某个正在做prefill的序列当前位于哪个stage上
        # =====
        self.enable_dynamic_chunking_schedule = (
            self.scheduler_config.enable_dynamic_chunking_schedule
        )
        # next four params apply only when using dynamic schedule
        self.low_chunk_size = self.scheduler_config.low_chunk_size
```

```

self.high_chunk_size = self.scheduler_config.high_chunk_size
self.chunk_schedule_max_tokens =
self.scheduler_config.chunk_schedule_max_tokens
self.chunk_schedule_stages = self.scheduler_config.chunk_schedule_stages

if self.enable_dynamic_chunking_schedule:
    assert self.chunk_schedule_stages > 0
    assert self.chunk_schedule_max_tokens > 0
    assert self.low_chunk_size % 32 == 0
    assert self.high_chunk_size % 32 == 0
    # 计算在动态变更chunk_size的情况下，我们可选的chunk_size列表（详情参见相关函数
    注释）

    self._chunk_sizes = self._compute_chunk_size_schedule()
    # 用于计算每个stage能处理的token数（详细解释见上）
    self._tokens_per_stage = int(
        np.ceil(self.chunk_schedule_max_tokens /
self.chunk_schedule_stages)
    )

def _compute_chunk_size_schedule(self):
    # =====
    # create num_steps equally spaced chunk sizes
    # between low_chunk_size and high_chunk_size
    #
    # self.low_chunk_size = 64
    # self.high_chunk_size = 256
    # self.chunk_schedule_stages = 5
    # 则chunk_sizes = [64, 108, 152, 196, 256]
    # 按照从大到小排序后 = [256, 196, 152, 108, 64]
    # =====
    chunk_sizes = np.linspace(
        self.low_chunk_size,
        self.high_chunk_size,
        self.chunk_schedule_stages,
        dtype=np.int32,
    )[:-1]

    # =====
    # 这里是调整每个备选的分块大小，让其能够被32整除
    # 这样做是考虑到tile-quantization effect，让gpu做gemm时的并行性能最大化
    # =====
    round_of_chunk_sizes = min(32, self.low_chunk_size)
    chunk_sizes = (
        np.round(chunk_sizes / round_of_chunk_sizes) * round_of_chunk_sizes
    )
    chunk_sizes = chunk_sizes.astype(np.int64).tolist()

    return chunk_sizes

def get_block_space_manager_class(self):
    return SarathiBlockSpaceManager

def _get_seq_next_num_prefill_tokens(
    self, seq: Sequence, num_batched_tokens: int
) -> int:
    """

```

```

对于一条还没做完prefill的seq，根据当前batch中已经存放的tokens数量，决定要送
这个seq的多少tokens去做prefill
"""

assert not seq.is_finished()

# =====
# 如果使用动态chunk_size的方法
# =====
if self.enable_dynamic_chunking_schedule:
    # =====
    # 先计算当前seq目前一共处理了多少prefill tokens，然后根据每个阶段其最多能处理
    # 的prefill tokens数量，确定它在第几阶段（stage）
    # =====
    request_stage_idx = int(
        np.ceil(seq.get_num_prompt_tokens_processed() //
self._tokens_per_stage)
    )
    # =====
    # 取出这个阶段的chunk_size
    # =====
    assert request_stage_idx < len(self._chunk_sizes)
    chunk_size = self._chunk_sizes[request_stage_idx]
    # =====
    # 如果没有使用动态变更chunk_size的策略，就用固定尺寸的chunk_size
    # （例如代码中的默认值512）
    # =====
else:
    chunk_size = self.chunk_size

# =====
# 对于这个正在做prefill的seq，确定它在下一次迭代中要送去做prefill的tokens数量。
# 这个数量 = min(该序列还没有做prefill的tokens数，batch中可用的prefill tokens配
额)
# =====
next_num_tokens = min(
    seq.get_prompt_len() - seq.get_num_prompt_tokens_processed(),
    chunk_size - num_batched_tokens,
)

return next_num_tokens

def _schedule(self) -> SchedulerOutputs:
    # Fix the current time.
    now = time.monotonic()

    running: List[Sequence] = [] # 应该是用来存放确定要被本轮调度的数据
    ignored_seq_ids: List[str] = []
    preempted_seq_ids: List[str] = []
    scheduled_seq_metadata_list: List[SequenceScheduleMetadata] = []

    num_batched_tokens: int = 0

    #####
    # Phase 1: Add existing running sequence groups to the batch.
    # There are two cases:
    # 1. The sequence group has incomplete prefill. The routine
    # remains identical to the one in sarathi scheduler for such sequences.

```

```

# 2. The sequence group has completed prefill. In this case, we need to
# check for memory availability for the next chunk of decode tokens, and
preempt
# some sequence groups if necessary. Note that, the preempted sequence
groups
# might belong to either of the two categories.
#####

# NOTE(woosuk): Preemption happens only when there is no available slot
# to keep all the sequence groups in the RUNNING state.
# In this case, the policy is responsible for deciding which sequence
# groups to preempt.

# =====
# 把self.running中的数据按照FCFS原则（先来到后到）进行排序
# =====
self.running = self.policy.sort_by_priority(now, self.running)

# in first pass process all the requests with prefill completed
# this allows us to accurately account for the number of decode tokens
running_prefills: List[Sequence] = []

# =====
# 先去看上一次iteration中被选中的序列
# =====
while self.running:
    seq = self.running.pop(0)
    # =====
    # 如果这个seq没有被暂停，那么就把它继续添加到本轮running队列中
    #
    # （上一轮调度结束后，所有running状态的序列都会被设置为pause状态，
    # 这里可以参考base_sequence_manager的on_step_completed函数，
    # 这个函数是对每轮调度结束后序列的状态和推理结果做处理），
    #
    # （当然也可能有别的条件会触发pause状态设置，这里没有看完全部源码，所以暂不知道）
    # =====
    if not seq.is_paused():
        running.append(seq)
        continue

    # =====
    # 如果这个seq还没有做完prefill，就把它添加到running_prefill的列表中
    # =====
    if not seq.prompt_processing_finished:
        running_prefills.append(seq)
        continue

    # =====
    # （走到这一步，剩下的都是上一次调度中处于decode阶段的seq了）
    # 如果现在没有足够的空间给处于decode阶段的seq做推理了
    # =====
    while not self.block_manager.can_append_slot():
        #
        =====

        # 如果self.running队列中有数据，就从running队列中抢占最晚到来的那个
        # sarathi中的抢占是直接做重计算，即把seq重新放回waiting队列中

```

```

#
=====
    if self.running:
        # Preempt the lowest-priority sequence groups.
        victim_seq = self.running.pop(-1)
        self._preempt(victim_seq)
        preempted_seq_ids.append(victim_seq.seq_id)
    #
=====
    # 如果self.running队列中已经没有数据了，就抢占当前seq
    #
=====

    else:
        # No other sequence groups can be preempted.
        # Preempt the current sequence group.
        self._preempt(seq)
        preempted_seq_ids.append(seq.seq_id)
        break

# =====
# 如果现在有足够空间给处于decode阶段的seq做推理
# =====
else:
    # 给decode阶段的seq分配KV cache空间，并将其添加到本轮的running队列中
    self._append_slot(seq)
    running.append(seq)
    # 当前batch的token数量 += 1
    num_batched_tokens += 1
    scheduled_seq_metadata_list.append(
        SequenceScheduleMetadata.from_sequence(seq)
    )

# =====
# 接下来处理上一次调度中没有做完prefill的seq
# 他们的KV cache空间肯定是够的，因为对于一个seq，我们在一开始是根据
# 它完整的prefill序列长度来分配KV cache，而不是根据prefill chunk大小分配
# KV cache。所以无论是那一轮iteration，我们都给这个seq的prefill留足了
# KV cache空间
# now add the requests with prefill incomplete
# the memory for all these prefills has already been allocated
# so we should be able to run all of them
# =====
for seq in running_prefills:
    assert not seq.prompt_processing_finished
    # =====
    # 计算对于这个seq，这次调度可以放多少tokens去做prefill
    # =====
    next_num_prefill_tokens = self._get_seq_next_num_prefill_tokens(
        seq, num_batched_tokens
    )

    # as long as the request could fit in the batch previously
    # it should be able to fit in the batch now
    # so in non-pipeline case this condition should always be false
    # however, in pipeline case, the grouping of requests can change
    # between different microbatches, so this is not guaranteed to be
always true

```

```

if next_num_prefill_tokens == 0:
    running.append(seq)
    continue

num_batched_tokens += next_num_prefill_tokens
scheduled_seq_metadata_list.append(
    SequenceScheduleMetadata.from_sequence(
        seq, prompt_chunk_len=next_num_prefill_tokens
    )
)
running.append(seq)

#####
# Phase 2: Add waiting (new) sequence groups to the batch.
# This routine is nearly-identical to the one in sarathi scheduler
# 在phase1中，我们遍历了上一个iteration的batch，来决定有哪些seq可以继续做
# 这一轮的推理。
# 在phase2中，我们去waiting队列中继续搜寻，看看是否有新请求能加入这一轮推理
# 也就是每次调度中，batch = 上一轮batch筛选后的结果 + waiting队列中筛选的结果
#####
# Optimization: We do not sort the waiting queue since the preempted
# sequence groups are added to the front and the new sequence groups
# are added to the back.
while self.waiting:
    seq = self.waiting[0]

    # This is required to handle benchmarking where we set request
arrival time ahead of time
    if seq.arrival_time > now:
        break

    if not self._check_request_prompt_length(seq):
        ignored_seq_ids.append(seq.seq_id)
        continue

    # =====
    # If the sequence group cannot be allocated, stop.
    # 直接用了vllm的allocate方法，即不是根据seq的prefill chunk大小
    # 预分配物理块的，而是直接根据整个seq的prefill大小分配物理块的
    # =====
    if not self.block_manager.can_allocate(seq):
        # this is different from vllm scheduler
        # even if we cannot allocate this sequence group
        # there might be other sequence groups that can be allocated
        break

    # The total number of sequences in the RUNNING state should not
    # exceed the maximum number of sequences.
    if len(running) >= self.scheduler_config.max_num_seqs:
        break

    # check if we can fit the prefill in the batch
    next_num_prefill_tokens = self._get_seq_next_num_prefill_tokens(
        seq, num_batched_tokens
    )

```

```

if next_num_prefill_tokens == 0:
    break

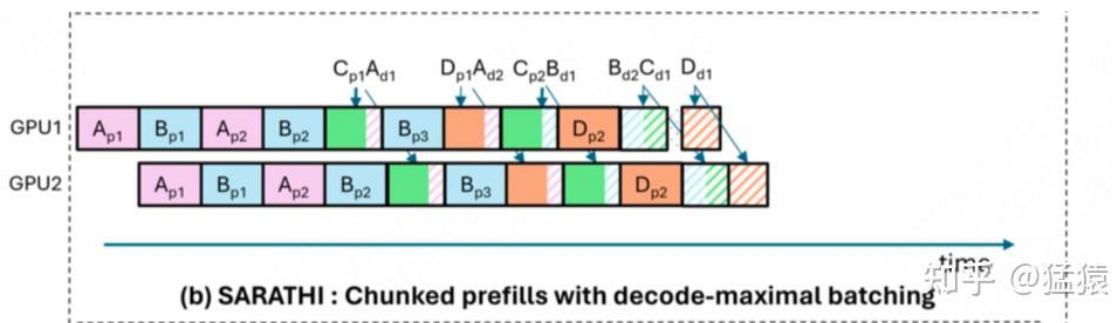
seq = self.waiting.pop(0)
self._allocate(seq) # 直接为完整的seq prefill（而不是chunk prefill）分配
KV cache空间
num_batched_tokens += next_num_prefill_tokens
scheduled_seq_metadata_list.append(
    SequenceScheduleMetadata.from_sequence(
        seq, prompt_chunk_len=next_num_prefill_tokens
    )
)
running.append(seq)

# make sure that prefills are at the start of the batch, so that we don't
violate assumptions
# made in the original vllm codebase
self.running = running

return SchedulerOutputs(
    id=self._iteration_id,
    ignored_seq_ids=ignored_seq_ids,
    preempted_seq_ids=preempted_seq_ids,
    scheduled_seq_metadata_list=scheduled_seq_metadata_list,
)

```

我们可以配合着下面这张图来解读源码：



总体来说，Sarathi的源码其实是基于vllm源码框架修改而来的（最新版本的vllm源码中也做了chunked-prefills的优化，等我有时间把这块写进vllm源码解读里）。注释中已经给出了所有的细节，这里额外强调几点：

- 当整个系统刚启动时，batch中只有做prefill的序列。这时走的是源码中从waiting队列里调度的逻辑。在sarathi中，我们是根据整个prefill的长度预先分配好KV cache空间（而不是根据prefill chunk长度来分配的）。这确保了在后面所有的iteration中，我们不用再操心这个batch中这条prefill序列的KV cache问题，它一定是留足了空间。
- sarathi提供了“固定”和“动态”两种chunk size策略：

- **在固定chunk_size策略中**，默认值为512。这是sarathi根据硬件和profiling实验计算出来的能最大化saturate gpu computation的单batch中的tokens数量。从源码中不难知道，在系统刚启动时，每个请求的头512个prefill tokens各组成一个batch（如上图所示），进行前向推理。
- **随着推理迭代的进行，陆续有请求完成了prefill，进入decode过程**，比如上图中产出了Ad1。那么根据源码，A所在的这个batch，此时要分配1配额的tokens给Ad1继续做decode。同时，它要去waiting队列中按FCFS（先到先服务）的原则找出请求C。由于batch总tokens配额是512，所以它切割了C的511个tokens装进这个新batch中，以此类推。
- **随着迭代的继续进行，这个batch中总有一些序列是在prefill中，有一些序列是在decode中**。每一次在做新的调度迭代时，**对于正在做decode的策略，我们会先检查当前是否有足够的KV cache空间留给他们做新一轮迭代**，如果没有的话就需要抢占decode序列（细节在源码注释中）。而对于这个batch的prefill序列，正如前文所说，当他进入这个batch的那一刻起，就已经给他分配了完整的KV cache空间，所以它无需再担心这点。
- 可能在你的印象中，固定大小chunked-prefills意味着每个batch中prefill tokens的数量是不变的，但是通过sarathi的源码解读，你可以发现，尽量保持不变的是batch中的总tokens配额，而prefill tokens数量是随着decode tokens的增减而变动的（只不过decode tokens的数量一般也不多，所以prefill tokens数量和整体batch tokens配额也不会相差很多）
- **在动态chunk_size策略中，我们希望对于一个请求，它的prefill tokens的数量能随着迭代次数的增加而减少，这主要是为了解决较长序列带来的影响。当一条prompt特别长时，它在每一次迭代中都会占据一定计算资源，导致历史累积的decode序列和新来的请求受到影响**。所以干脆，对于进入这个batch中的请求，在一开始我们多给它一些prefill tokens配额，然后随着迭代次数的增加，递减这个配额，降低它对别人的影响。

论文中其实做了非常多关于性能的实验，篇幅原因这里不再一一给出，大家可以自行阅读论文。

3.6 chunked-prefills VS 分离式推理架构

通过以上的介绍，你已经知道，在使用chunked-prefills的策略下，通过合理划分prefill tokens和decode tokens比例，最大化利用好gpu，似乎也能同时保全TTFT和TPOT/TBT。那么在这样的前提下，分离式推理架构还有什么优势呢？

其实如果想更好回答这一点，最好的方式是做消融实验并分析。我没有做过相关的实验，所以只能从原理上给出我自己的一些猜想：

我觉得最主要的一点，是chunked-prefills可能还没有完全实现在达到TPOT/TBT SLO的情况下，最大化prefill阶段对GPU FLOPS的利用率（MFU）。我们从3.3的分析中可以发现，chunked-prefills是会产生额外开销的（overhead），这个开销不仅体现在他需要额外读取KV cache，还体现在prefill chunk

size的设定上。我们知道GPU的矩阵计算是存在tile-quantization effect的，也即矩阵是被切分成tiles后送到thread blocks上去做并行计算的。如果你的矩阵尺寸是tiles尺寸的整数倍数，那么就可以最大化并行计算，否则那些除不尽的部分就可能产生额外的开销（Sarathi做过相关实验，在tile_size = 128的前提下，257的矩阵尺寸比256的矩阵尺寸产生的prefill time多了32%）。而在chunk-prefill中，我们只是用profiling估算出在特定设备上一个batch的最大tokens配额而已，这些tokens包括prefill和decode。这个size是对整体的，而不是单独对prefill或decode的。所以仍然存在prefill阶段无法最大化MFU的可能。

第二个，也是从无法最大化prefill MFU上衍生出来的问题：chunked-prefills对长序列的处理可能还差强人意。从3.5的源码解读中，我们发现在chunked-prefills中，长序列持久地占据着KV cache的存储空间以及gpu的计算资源。尽管我们可以采用动态减少chunk_size的办法，来减少长序列的影响。但是——一来，这个chunk_size递减的策略要怎么设置更合理（而不是像3.5源码中那样可能是自己凭经验拍了一个），还有待研究。二来即使是实现了更好的chunk_size递减策略，但它却使得长序列的TTFT变大了，同样影响用户体验。

所以，基于以上这些对chunked-prefills策略缺陷的猜想，或许使用分离式架构，对prefill阶段独立开发一套策略，可能可以更加针对性地解决以上问题。当然，这也取决于各策略的具体实现、业务场景和真实的实验效果。

四、参考

- 1、<https://arxiv.org/abs/2306.02707>
- 2、<https://arxiv.org/abs/2308.16369>
- 3、<https://arxiv.org/abs/2403.02310>
- 4、<https://github.com/microsoft/sarathi-serve>
- 5、vllm、FasterTransformer等相关资料，不一一列举